CS6650 Assignment2 Zhenyu Wang & Xuming Zhuge

Project Link: https://github.khoury.northeastern.edu/yuchenwang00/CS6650-Assignment2-LoadingTest

Client Design Overview

1. System Architecture

The load-testing client consists of three programs:

- SingleThreadMain
- ClientPart1Main
- ClientPart2Main

The purpose of the system is to generate 200,000 POST requests to a Product microservice deployed on AWS ECS and evaluate its scalability and performance under concurrent load.

The architecture separates:

- Data generation
- Network request execution
- Logging
- Statistics computation

This separation minimizes synchronization overhead and maximizes concurrency.

2. Single-Thread Baseline

The SingleThreadMain program sends 10,000 POST requests sequentially.

For each request:

- Timestamp is taken before sending.
- Timestamp is taken after response.
- Latency is recorded.

It calculates:

- Average latency
- Minimum latency
- Maximum latency
- Throughput (requests/second)

Measured Results:

- Average latency ≈ 78 ms
- Throughput ≈ 12.6 rps

Predicted 8-thread throughput:

$8 \times 12.6 \approx 101$ \text{ rps}

This provides a theoretical scaling baseline.

```
====== Single Thread Result ======
total=10000
success=10000
failed=0
wallTimeSeconds=791.481
avgLatencyMs=78.62
minLatencyMs=67
maxLatencyMs=396
throughputRps=12.634542079974125
```

3. Client Part 1 – Multithreaded Load Generator

Design Goals

- Upload 200,000 products as fast as possible.
- Achieve near-linear scalability.
- Terminate cleanly.

Concurrency Model

- ExecutorService with fixed thread pool.
- Each request runs as an independent task.
- Each task creates its own ApiClient instance to ensure thread safety.
- LongAdder is used for success/failure counters to avoid contention.
- A progress thread prints every 1000 completed requests.

Execution Flow

1. Submit 200,000 tasks to thread pool.
2. Each task:
   - Creates Product object
   - Sends POST request
   - Updates success/failure counter
3. Main thread waits via CountDownLatch.
4. Wall time and throughput calculated at completion.

Results (8 threads)

- Throughput ≈ 99 rps
- Theoretical prediction ≈ 101 rps

Deviation < 2%

Conclusion:

The client demonstrates near-linear scaling.
No significant client-side bottlenecks were observed.

```
===== Client Part 1 Result =====
threads=8
total=200000
success=200000
failed=0
wallTimeSeconds=2018.958
throughputRps=99.06100077366641
```

4. Client Part 2 – Instrumented Client

Part 2 extends Part 1 by collecting detailed per-request metrics without affecting upload speed.

Additional Design Components

- Dedicated Product Generator Thread
- BlockingQueue<Product> to decouple generation and posting
- Multiple PostingWorker threads consuming from queue
- BlockingQueue<LogRecord> for asynchronous logging
- Dedicated CSV Writer Thread

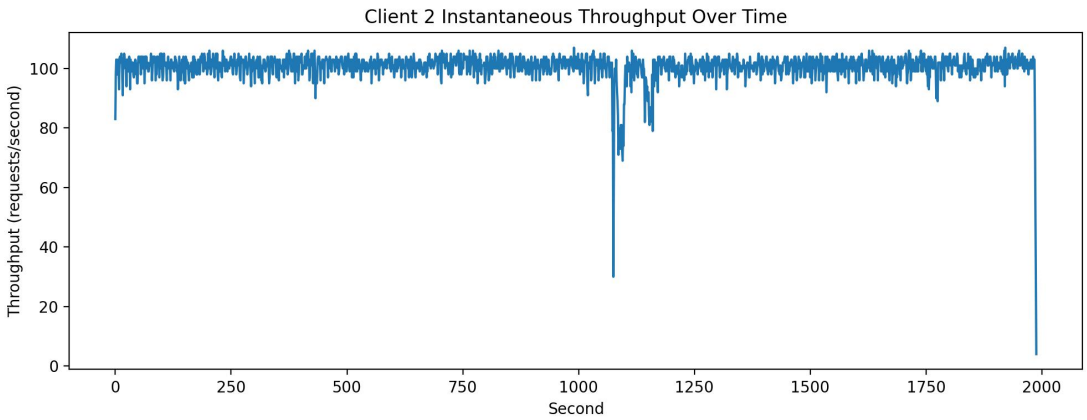
Data Flow

Product Generator
↓
BlockingQueue<Product>
↓
PostingWorker Threads
↓
BlockingQueue<LogRecord>
↓
CSV Writer
↓
StatsCalculator

This design ensures:

- Posting threads never block on product generation.
- Logging does not interfere with request throughput.
- Statistics are computed after execution.

```
====== Client Part 2 Result ======
threads=8
total=200000
success=200000
failed=0
wallTimeSeconds=1988.787
throughputRps=100.56381100640742
meanLatencyMs=79.44369918274373
medianLatencyMs=77.5454795
p99LatencyMs=150.800334
minLatencyMs=66.777083
maxLatencyMs=524.648167
Wrote throughput_over_time.csv
```



5. Performance Metrics Collected

After execution completes, the client calculates:

- Mean latency
- Median latency
- p99 latency
- Minimum latency
- Maximum latency
- Throughput
- Instantaneous throughput over time

Measured Results (8 threads):

- Mean latency ≈ 79 ms
- Median latency ≈ 78 ms
- p99 latency ≈ 150 ms
- Throughput ≈ 100 rps

Throughput closely matches predicted performance.

6. Why HTTP 201 Instead of 200?

According to REST semantics:

- 200 OK indicates a successful request.
- 201 Created indicates a new resource was created.

Since the POST endpoint creates a Product resource, the correct REST-compliant response is HTTP 201.

7. Scalability Analysis

Single-thread throughput ≈ 12.6 rps
8-thread throughput ≈ 99 rps

Theoretical: 101 rps
Measured: 99 rps

The small deviation is attributed to:

- Network latency
- Thread scheduling overhead
- AWS infrastructure variability

Conclusion:

The system achieves near-linear scalability and demonstrates correct concurrent design.